VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA0 - HK241

Cuối kì

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 12/2024

Mục lục

1 BTL1_HK241 **2**

1 BTL1_HK241

Cho một class abstract - Trả Lời 15 câu hỏi sau

```
template<class T>
class IList{
public:
    virtual ~IList(){};
    virtual void    add(T e)=0;
    virtual void    add(int index, T e)=0;
    virtual T       removeAt(int index)=0;
    virtual bool    removeItem(T item, void (*removeItemData)(T)=0)=0;
    virtual bool    empty()=0;
    virtual int     size()=0;
    virtual void    clear()=0;
    virtual T&      get(int index)=0;
    virtual int     indexOf(T item)=0;
    virtual bool    contains(T item)=0;
    virtual string  toString(string (*item2str)(T&)=0 )=0;
};
```

Class XArrayList thừa kế từ IList - Trả Lời 5 câu hỏi sau

```
template <class T>
class XArrayList : public IList<T> {
 public:
  class Iterator;  // forward declaration

 protected:
  T *data;
  int capacity;
  int count;
  bool (*itemEqual)(T &lhs, T &rhs);
  void (*deleteUserData)(XArrayList<T> *);

 public:
  XArrayList(void (*deleteUserData)(XArrayList<T> *) = 0,
             bool (*itemEqual)(T &, T &) = 0, int capacity = 10);
  XArrayList(const XArrayList<T> &list);
  XArrayList<T> &operator=(const XArrayList<T> &list);
  ~XArrayList();

  // todo FUCNTION Inherit

  Iterator begin() { return Iterator(this, 0); }
  Iterator end() { return Iterator(this, count); }

protected:
  static bool equals(T &lhs, T &rhs, bool (*itemEqual)(T &, T &)) {
    if (itemEqual == 0)
      return lhs == rhs;
    else
      return itemEqual(lhs, rhs);
  }
  void checkIndex(int index);      // check validity of index for accessing
  void ensureCapacity(int index);  // auto-allocate if needed
  void copyFrom(const XArrayList<T> &list);
  void removeInternalData();
```

1. Điền code vào vị trí sau **/*Code 1*/**

```
template <class T>
void XArrayList<T>::add(T e) {
  this->ensureCapacity(this->count);
  this->data[/*Code 1*/] = e;
}
```

a) this.count b) this->count c) this->count++ d) ++this->count

2. Đầu là cách dùng foreach or đúng trong các câu sau khi duyệt qua tất cả phần tử

```
(1) for (auto it : list) {}
(2) for (auto it = list) {}
(3) for (auto it = list.begin(); it != list.end(); it++) {}
(4) for (auto it = list; it < list.end(); i--) {}
```

a) (1) b) (2) và (3) c) (1) và (2) d) Tất cả đều đúng

3. Điền code vào vị trí sau **/*Code 1*/**

```
template <class T>
T &XArrayList<T>::get(int index) {
  if (/*Code 1*/) {
    throw std::out_of_range("Index is out of range!");
  }

  return data[index];
}
```

a) index < 0 || index >= this->count b) index < 0 && index >= this->count
c) index <= 0 || index > this->count d) index <= 0 && index > this->count

4. Điền code vào vị trí sau **/*Code 1*/**

```
template <class T>
int XArrayList<T>::indexOf(T item) {
  for (int i = 0; i < this->count; ++i) {
    if (/*Code 1*/) {
      return i;
    }
  }

  return -1;
}
```

a) equals(data[i], item, itemEqual)
b) XArrayList::equals(data[i], item, itemEqual)
c) equals(data[i-1], item, itemEqual)
d) XArrayList::equals(data[i-1], item, itemEqual)

5. Điền code vào vị trí sau **/*Code 1*/ và /*Code 2*/ lần lượt là**

```cpp
template <class T>
bool XArrayList<T>::removeItem(T item, void (*removeItemData)(T)) {
  for (int i = 0; i < this->count; i++) {
    if (XArrayList::equals(data[i], item, itemEqual)) {
      T temp = /*Code 1*/;
      if (removeItemData) {
        /*Code 2*/
      }
      return true;
    }
  }

  return false;
}
```

a) removeAt(i) và removeItemData(data[i]);
b) delete data[i] và removeItemData(data[i]);
c) removeAt(i) và removeItemData(data);
d) delete data[i] và removeItemData(data[i]);

Class DLinkedList thừa kế từ IList (private trong khóa cuối kì) - Trả Lời 10 câu hỏi sau

```
template <class T>
class DLinkedList : public IList<T> {
 protected:
  Node *head;  // this node does not contain user's data
  Node *tail;  // this node does not contain user's data
  int count;
  bool (*itemEqual)(T &lhs, T &rhs);
  void (*deleteUserData)(DLinkedList<T> *);

 public:
  DLinkedList(void (*deleteUserData)(DLinkedList<T> *) = 0,
              bool (*itemEqual)(T &, T &) = 0);
  DLinkedList(const DLinkedList<T> &list);
  DLinkedList<T> &operator=(const DLinkedList<T> &list);
  ~DLinkedList();

  // TODO Inherit IList

  Iterator begin() { return Iterator(this, true); }
  Iterator end() { return Iterator(this, false); }
  BWDIterator bbegin() { return BWDIterator(this, true); }
  BWDIterator bend() { return BWDIterator(this, false); }

 protected:
  static bool equals(T &lhs, T &rhs, bool (*itemEqual)(T &, T &));
  void copyFrom(const DLinkedList<T> &list);
  void removeInternalData();
  Node *getPreviousNodeOf(int index);

 public:
  class Node {
   public:
    T data;
    Node *next;
    Node *prev;
    friend class DLinkedList<T>;
   public:
    Node(Node *next = 0, Node *prev = 0);
    Node(T data, Node *next = 0, Node *prev = 0);
  };
  class Iterator {
   private:
    DLinkedList<T> *pList;
    Node *pNode;

   public:
    Iterator(DLinkedList<T> *pList = nullptr, bool begin = true);
    Iterator &operator=(const Iterator &iterator);
    bool operator!=(const Iterator &iterator) const;
    Iterator &operator++();
  };

  class BWDIterator {
   private:
```

```
    DLinkedList<T> *pList;
    Node *pNode;

   public:
    BWDIterator(DLinkedList<T> *pList = nullptr, bool begin = true);
    BWDIterator &operator=(const BWDIterator &iterator);
    T &operator*();
    bool operator!=(const BWDIterator &iterator) const;
    BWDIterator &operator++();
  };
};
```

Trả Lời 5 câu hỏi sau

```
template <typename DType, typename LType>
class Dataset {
 private:
 public:
  Dataset() {};
  virtual ~Dataset() {};

  virtual int len() = 0;
  virtual DataLabel<DType, LType> getitem(int index) = 0;
  virtual xt::svector<unsigned long> get_data_shape() = 0;
  virtual xt::svector<unsigned long> get_label_shape() = 0;
};

template <typename DType, typename LType>
class DataLoader {
 public:
  class Iterator;

 private:
  Dataset<DType, LType>* ptr_dataset;
  int batch_size;
  bool shuffle;
  bool drop_last;

 public:
  DataLoader(Dataset<DType, LType>* ptr_dataset, int batch_size,
  bool shuffle = true, bool drop_last = false);
  virtual ~DataLoader();
  Iterator begin();
  Iterator end();

  class Iterator {
        //TODO
  };
};
```

1. Để triển khai được Iterator thì bắt buộc có những hàm nào bên dưới đếm số lượng

   ```
   (1) Iterator& operator=
   (2) Iterator()
   (3) Iterator& operator++()
   (4) Iterator operator++(int)
   (5) bool operator!=(const Iterator& other)
   (6) Batch<DType, LType> operator*() const
   (7) Iterator& operator--()
   ```

   a) 3 b) 4 c) 5 d) 6

2. Giả sử bạn tạo một đối tượng DataLoader với constructor sau và Trong trường hợp này, đối tượng DataLoader sẽ chia dữ liệu thành bao nhiêu batch?

   ```
   DataLoader<int, float> loader(dataset, 32, true, false);
   ```

   a) 32
   b) Số lượng mẫu chia cho 32 (với điều kiện len(dataset) % 32 != 0, có thể bỏ qua mẫu cuối)
   c) Tổng số lượng mẫu trong dataset
   d) Số lượng mẫu chia cho 32, bao gồm tất cả các batch kể cả batch cuối nếu không đủ 32 mẫu

3. Giả sử bạn tạo một đối tượng DataLoader với constructor sau và đối tượng DataLoader sẽ chia dữ liệu thành bao nhiêu batch nếu số lượng mẫu trong dataset là 100?

```
DataLoader<int, float> loader(dataset, 32, true, false);
```

   a) 3 batch (100 mẫu chia cho 32 mẫu mỗi batch)
   b) 4 batch (100 mẫu chia cho 32 mẫu mỗi batch, bỏ qua batch cuối không đủ)
   c) 3 batch (bao gồm một batch có 4 mẫu còn lại)
   d) 4 batch (bao gồm cả batch cuối nếu không đủ 32 mẫu)

4. Khi bạn sử dụng DataLoader để load dữ liệu từ một Dataset trong môi trường học máy, điều gì xảy ra khi bạn thay đổi giá trị của shuffle từ true thành false trong constructor của DataLoader?

```
DataLoader<int, float> loader(dataset, 32, true, false);
```

   a) Dữ liệu sẽ được lấy theo thứ tự ngẫu nhiên từ Dataset, không phụ thuộc vào giá trị của shuffle
   b) Dữ liệu sẽ luôn được lấy theo thứ tự ngẫu nhiên từ Dataset khi shuffle = true, và theo thứ tự ban đầu của Dataset khi shuffle = false
   c) DataLoader không có tác dụng nếu shuffle = false và sẽ không trả về dữ liệu
   d) Dữ liệu sẽ luôn được lấy theo thứ tự ban đầu của Dataset, không liên quan đến giá trị của shuffle

5. Tại sao DataLoader trong học máy lại chia dữ liệu thành các batch thay vì sử dụng toàn bộ dữ liệu một lần trong mỗi lần huấn luyện?

   a) Để giảm thiểu bộ nhớ cần thiết và giúp xử lý dữ liệu lớn mà không làm tràn bộ nhớ
   b) Để tăng tốc quá trình huấn luyện bằng cách xử lý từng phần dữ liệu một cách song song
   c) Để đảm bảo mô hình được huấn luyện theo cách tối ưu và tránh overfitting
   d) Để kiểm soát số lần cập nhật gradient và tăng tính chính xác của các phép toán học